Compte Rendu – TP 1

Application Monolithique avec Spring Boot

Gestion de Produits Simple

Réalisé par : Assamri Salma

Année universitaire : 2025 / 2026

Encadré par : Pr. BE ELBAGHAZAOUI

Introduction

Ce TP 1 a pour objectif de mettre en place une application monolithique simple avec Spring Boot. Il s'agit de développer une API REST complète permettant les opérations CRUD (Create, Read, Update, Delete) sur une entité Produit, en appliquant l'architecture en couches.

Objectifs du TP :

- Mettre en place une application monolithique simple avec Spring Boot
- Développer une API REST complète permettant les opérations CRUD
- Appliquer l'architecture en couches : entité, repository, service et contrôleur
- Découvrir l'utilisation de RestRepository pour générer automatiquement des endpoints REST
- Analyser les avantages et les limites d'une architecture monolithique

Outils Utilisés :

Les outils utilisés dans ce TP sont : JDK 17+, Maven, IntelliJ IDEA, MySQL, et Postman pour les tests des endpoints REST.

Étape 1 : Création du Projet Spring Boot

Le projet est initialisé via Spring Initializr (https://start.spring.io) avec les dépendances : Spring Web, Spring Data JPA, MySQL Driver, Lombok et Validation.

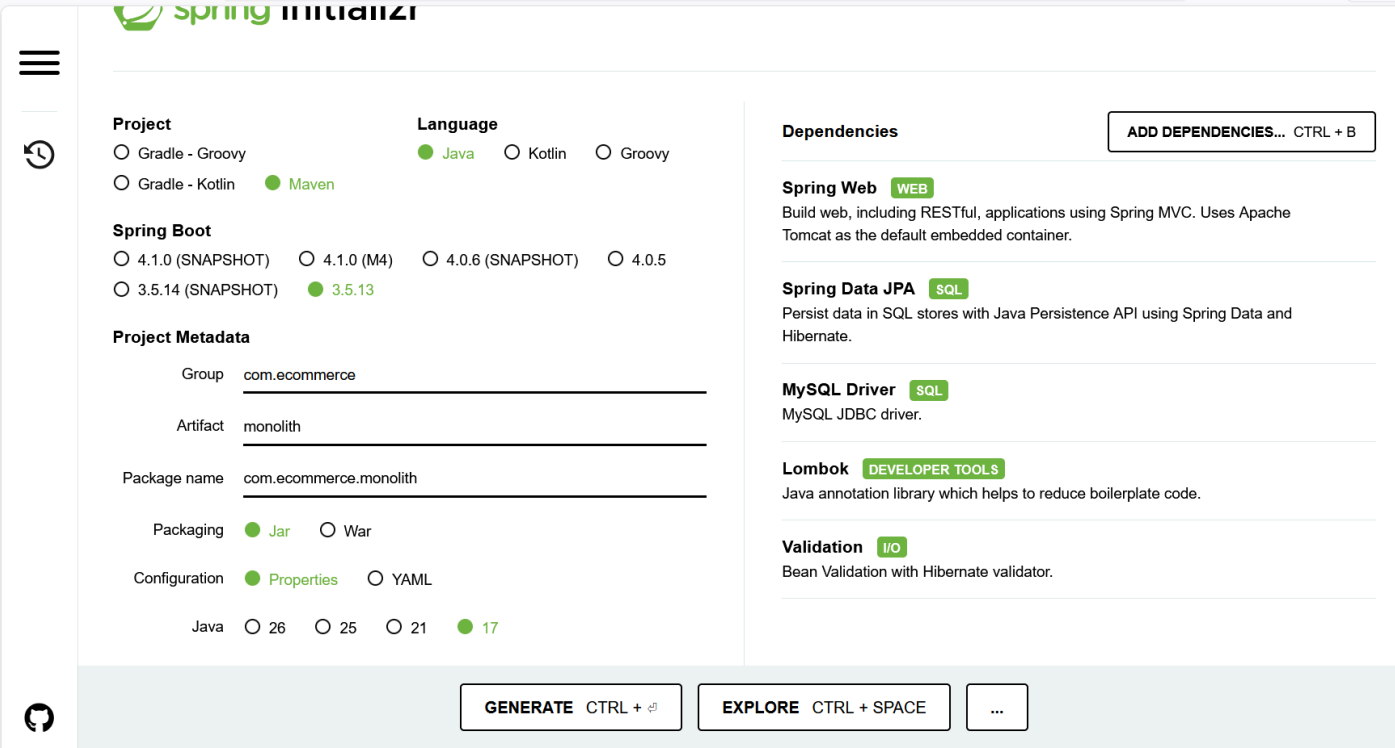

Figure 2 – Initialisation du projet Spring Boot avec Spring Initializr : configuration des dépendances (Spring Web, JPA, MySQL, Lombok, Validation)

Étape 2 : Création de la Base de Données MySQL

La base de données 'ecommerce' est créée dans MySQL. Le fichier application.properties est configuré pour établir la connexion avec la base de données locale.

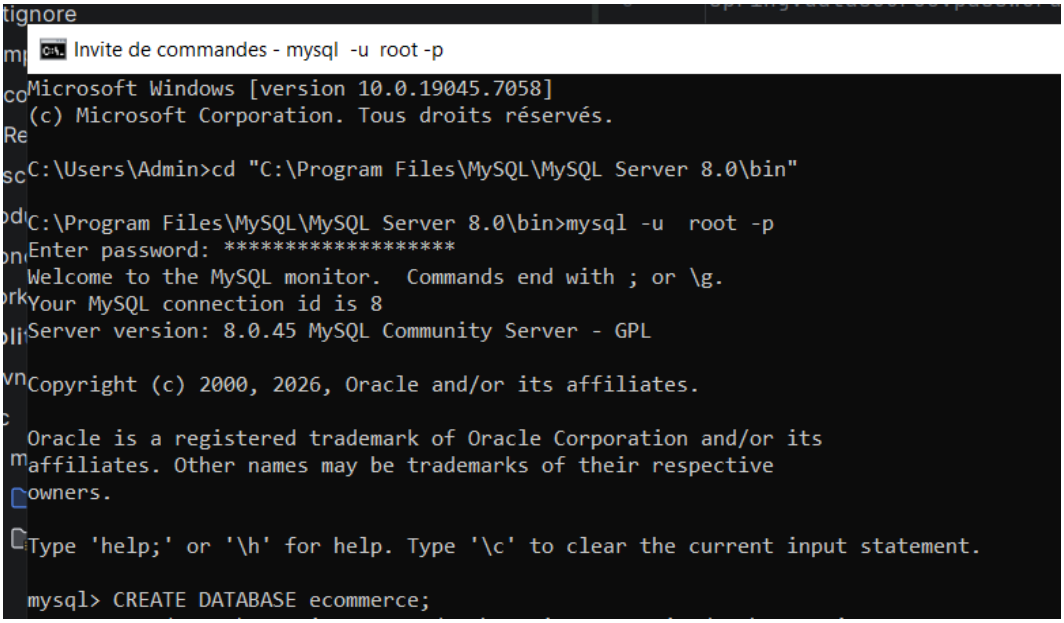

Figure 3 – Création de la base de données MySQL et configuration du fichier application.properties avec l'URL de connexion, username et password

Étape 3 : Architecture en Couches

L'application suit une architecture en couches claire avec : l'entité Product (model), le repository JPA, le service métier, et le controller REST exposant les endpoints sur /api/products.

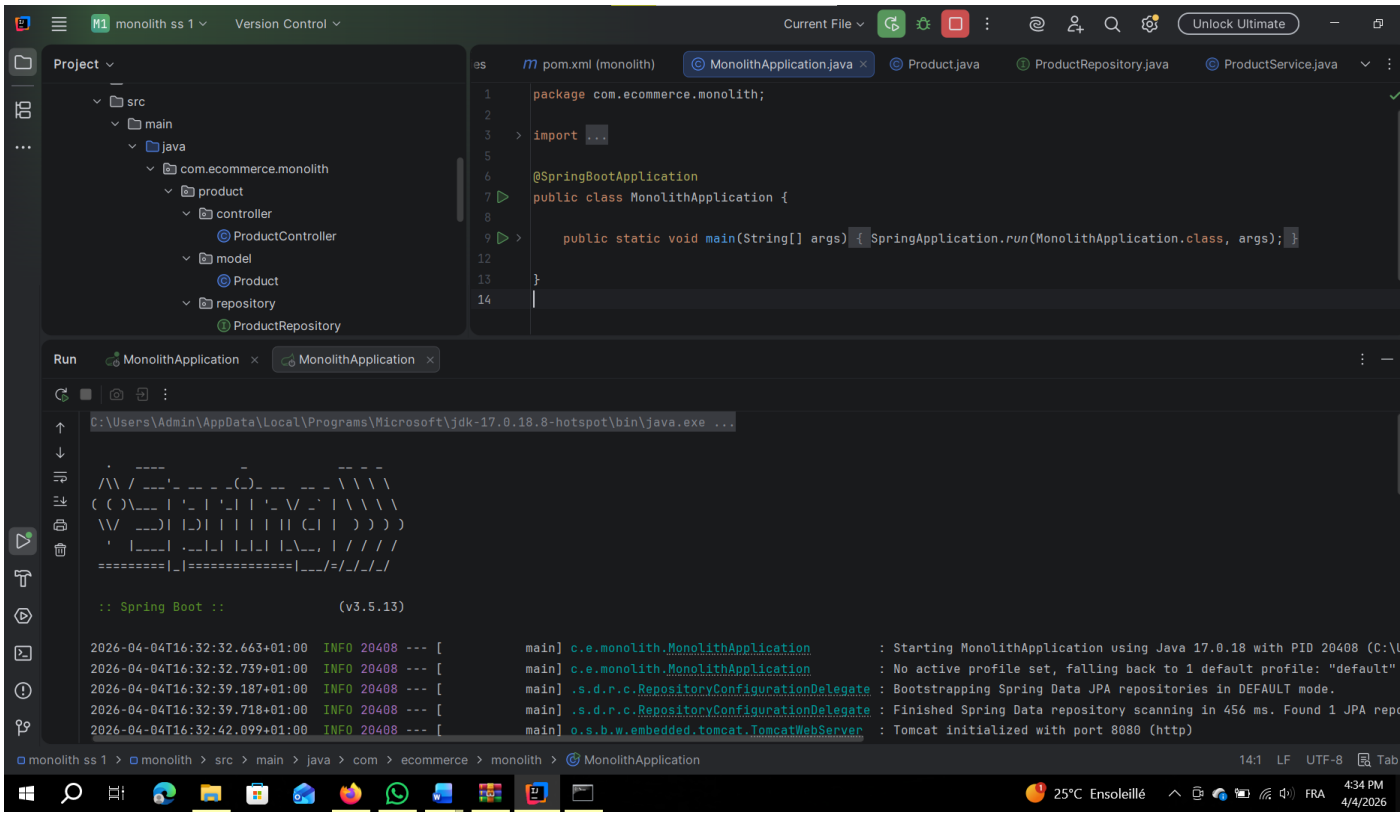

Figure 4 – Structure complète du projet dans IntelliJ avec les packages product.model, product.repository, product.service et product.controller

Étape 4 : Tests avec Postman

Les quatre opérations CRUD sont testées avec Postman sur les endpoints de l'API REST.

POST – Créer un produit

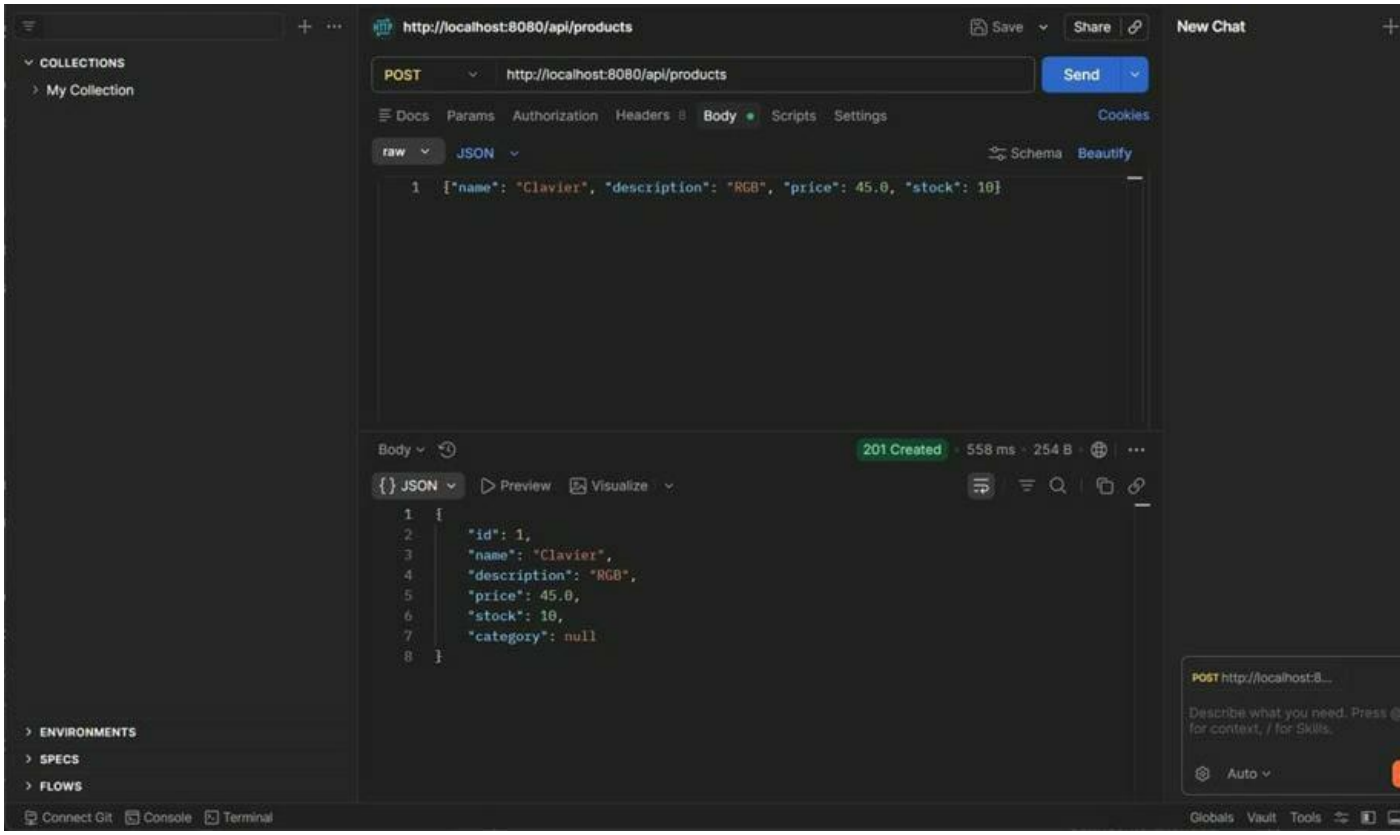

Figure 5 – Test POST /api/products : création d'un nouveau produit avec les champs name, description, price et stock. Réponse HTTP 201 Created

GET – Récupérer les produits

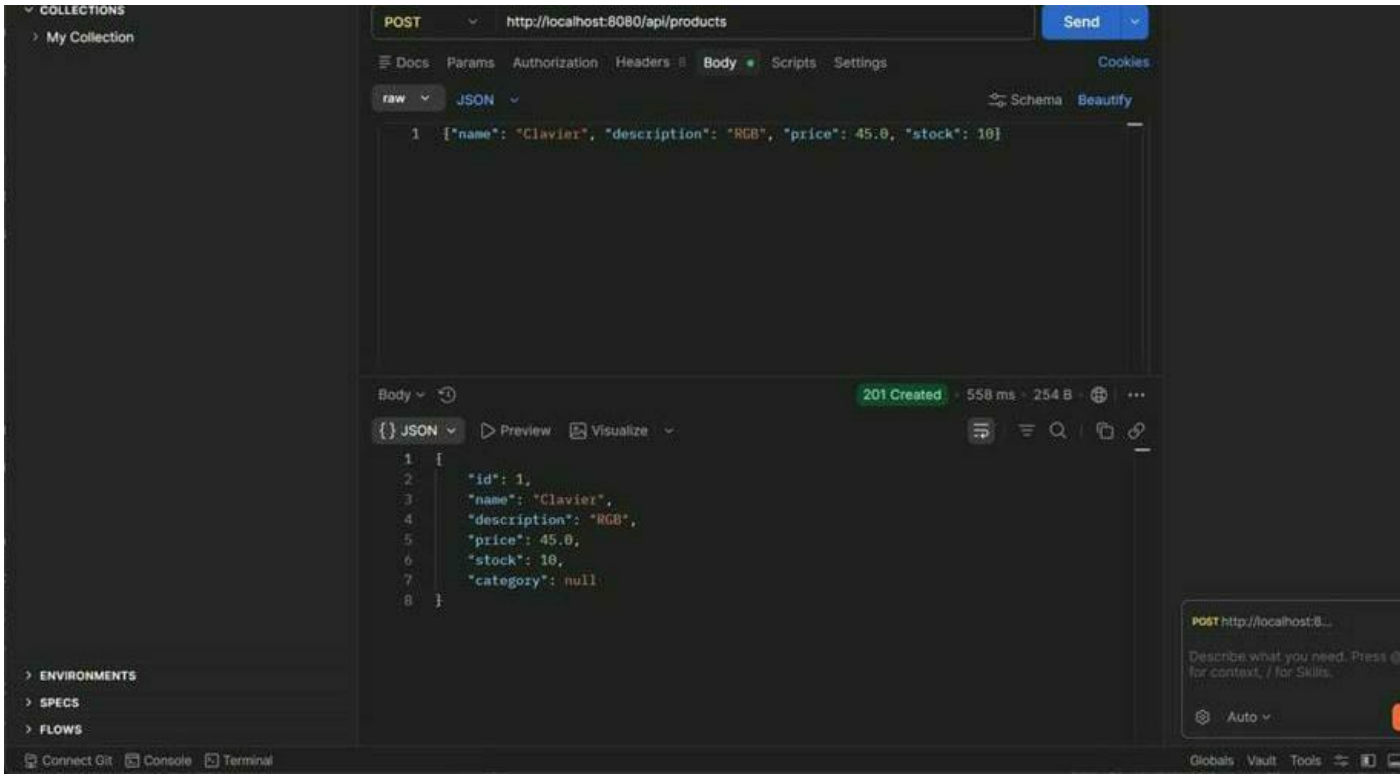

Figure 6 – Test GET /api/products : récupération de la liste complète des produits. Réponse HTTP 200 OK avec le tableau JSON

PUT – Modifier un produit

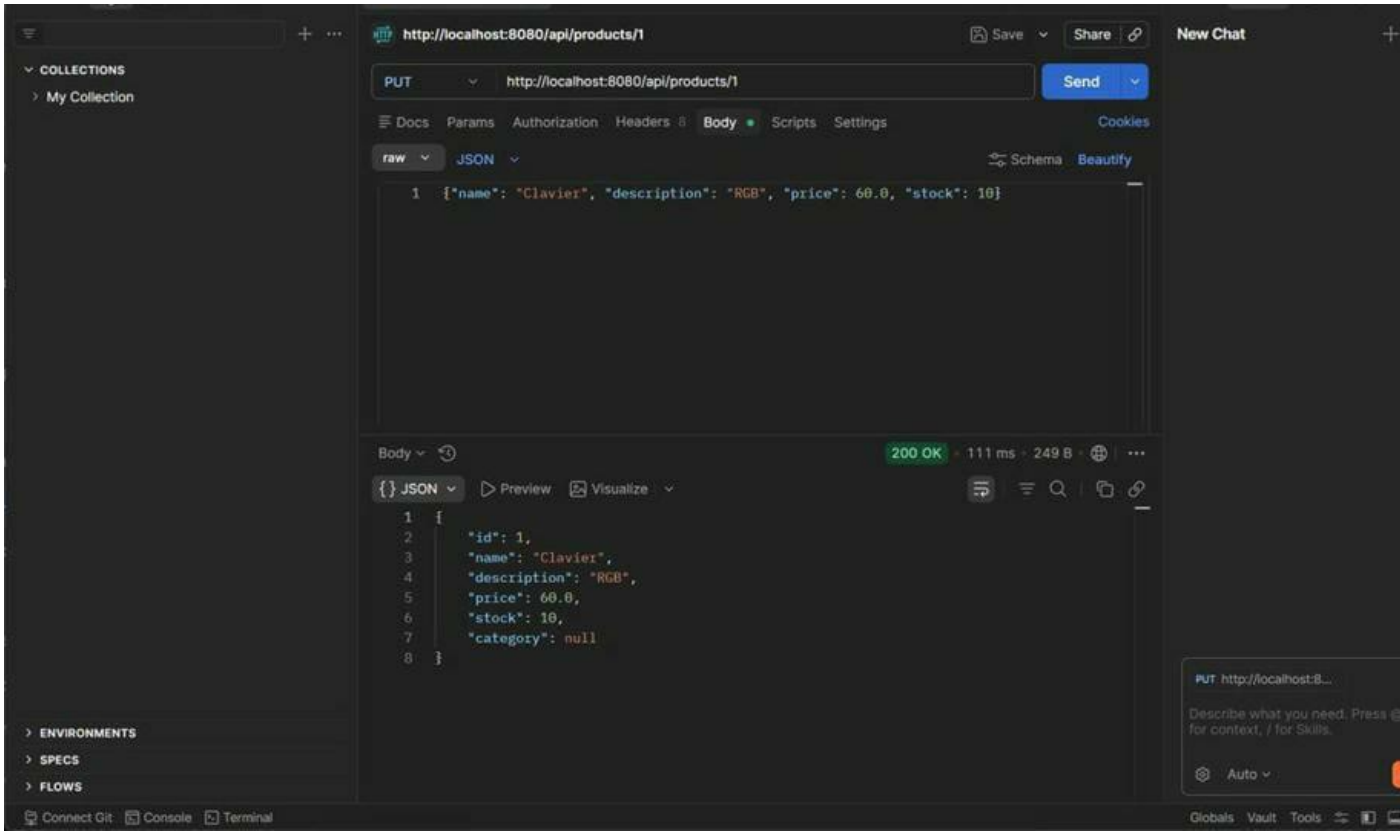

Figure 7 – Test PUT /api/products/{id} : modification d'un produit existant. Réponse HTTP 200 OK avec les données mises à jour

DELETE – Supprimer un produit

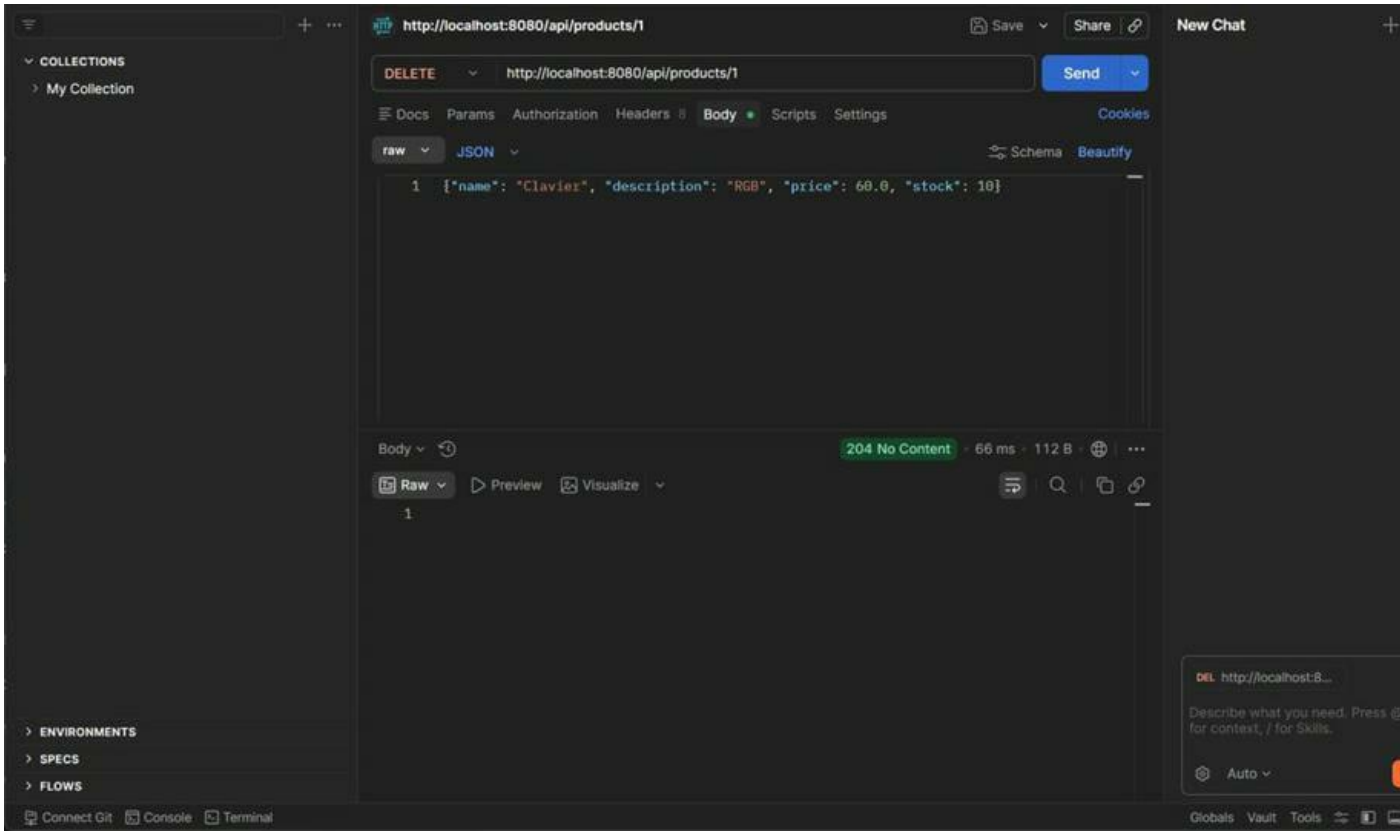

Figure 8 – Test DELETE /api/products/{id} : suppression d'un produit. Réponse HTTP 204 No Content

Exercice : Entité Category

L'exercice consiste à ajouter une entité Category et créer la relation ManyToOne avec Product, puis tester avec RestRepository.

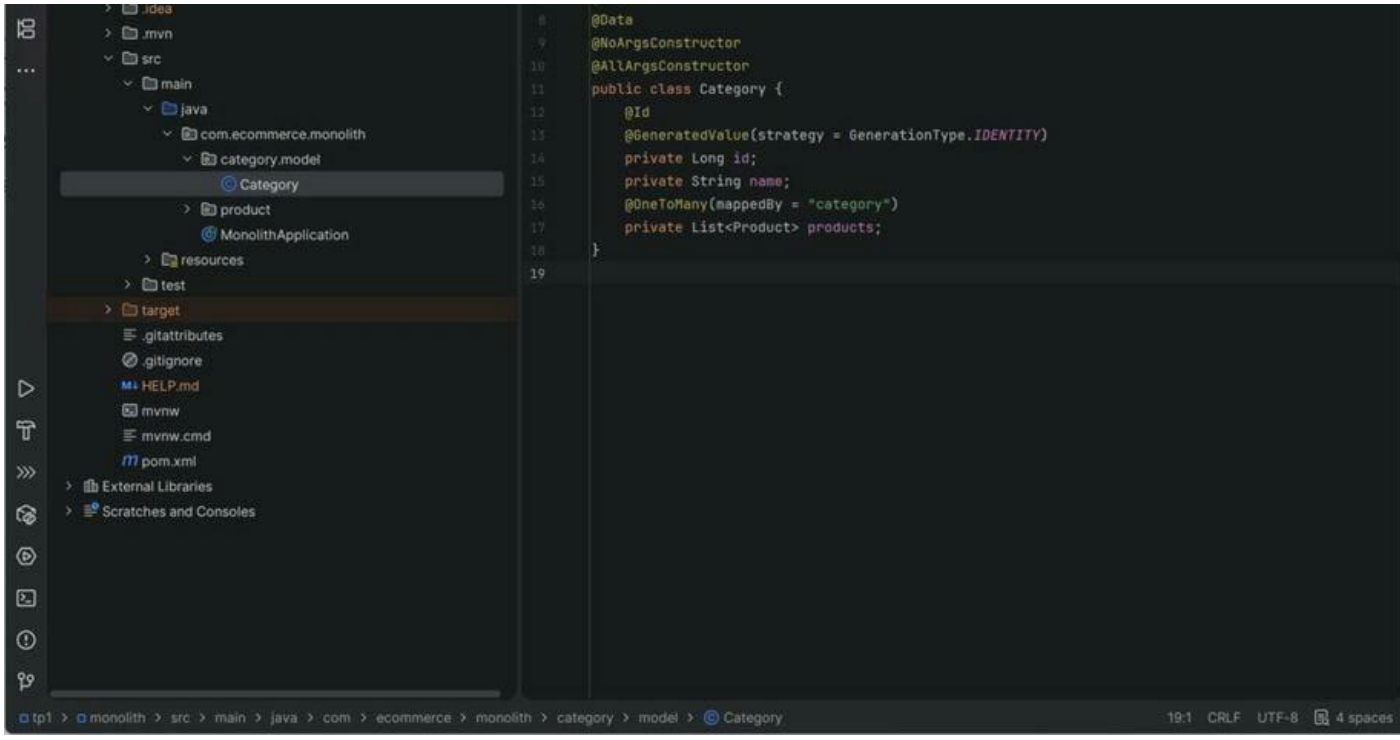

Figure 9 – Exercice : ajout de l'entité Category avec la relation @ManyToOne vers Product, testé avec RestRepository

Conclusion

Ce TP 1 a permis de maîtriser les bases du développement d'une application monolithique avec Spring Boot. L'architecture en couches (Entity, Repository, Service, Controller) a été appliquée avec succès, et les quatre opérations CRUD ont été validées avec Postman.

Points clés réalisés :

1. Création du projet Spring Boot avec MySQL
2. Implémentation de l'entité Product avec JPA et Lombok
3. Développement des couches Repository, Service et Controller
4. Tests complets des endpoints REST avec Postman (GET, POST, PUT, DELETE)
5. Exercice : relation ManyToOne entre Product et Category